

Draya Platform — API Contract Documentation

Purpose: Freeze the interface between backend, frontend, and mobile teams so all three can build in parallel against a shared, versioned contract.

Base URL: `https://api.draya.app/api/v1` **Format:** JSON over HTTPS (`multipart/form-data` only for file upload endpoints)

0. Revision Notes

0.1 — Payments Added

This revision adds classroom payments via Paymob. Two assumptions are baked into the design below (flag if wrong, both are isolated to Module 3A and easy to change):

- Draya is the merchant of record — a single platform-level Paymob integration; teachers don't connect their own merchant accounts. Draya deducts a configurable commission and pays teachers out periodically (Module 3A, payout endpoints).
- Payment is one-time per classroom enrollment, not a recurring subscription — a student pays once to join, access doesn't expire or renew.

Golden rule for this module: the client (web/mobile) is never trusted to report "payment succeeded." Only a signature-verified, server-to-server webhook from Paymob is allowed to create an Enrollment row for a paid classroom.

0.2 — Password Reset Added

This revision adds a self-service password reset flow (Module 2, endpoints 2.7-2.8) that was missing from the original contract. No other auth/session endpoints changed.

0.3 — Ownership Error Code Clarified

Section 1.1 previously read "403, not 404... actually 404 is returned" — a leftover drafting contradiction. This revision removes the ambiguity: **cross-tenant ownership violations always return 404**, matching Section 1.4's existing rule ("never leak existence"). No behavior changed, only the wording.

1. Conventions (read this before any module)

1.1 Authentication

- Scheme: JWT Bearer, issued by ASP.NET Core Identity.
- Header required on every endpoint except `/auth/register/*`, `/auth/login`, and `/auth/password-reset/*`.
- Access token TTL: 60 minutes. Refresh token TTL: 14 days (rotated on use).
- Token claims: `sub` (UserId, GUID), `role` (`Teacher` | `Student` | `SuperAdmin`), `email`, `exp`, `iat`.

- Every endpoint below states which role(s) may call it. Anything not listed as SuperAdmin-only but touching a Teacher's own data is scoped server-side by `sub == TeacherId` — a Teacher can never pass another teacher's `classroomId` and get data back. **This always returns 404, not 403** — returning 403 would itself confirm the resource exists, which leaks information about another tenant's data. See 1.4 for the full error code table.

```
Authorization: Bearer <access_token>
```

1.2 Standard Headers

Header	Required	Notes
<code>Authorization</code>	Yes (except auth endpoints)	<code>Bearer <token></code>
<code>Content-Type</code>	Yes	<code>application/json</code> or <code>multipart/form-data</code> for uploads
<code>Accept-Language</code>	Optional	<code>ar</code> or <code>en</code> , defaults to <code>ar</code> — affects AI-generated content language, not error messages
<code>X-Request-Id</code>	Optional	Client-generated GUID for tracing; echoed back in response

1.3 Pagination

All list endpoints accept `?page=1&pageSize=20` (default `pageSize=20`, max `100`) and return:

```
json
{
  "items": [ ... ],
  "page": 1,
  "pageSize": 20,
  "totalCount": 137,
  "totalPages": 7
}
```

1.4 Error Envelope

Every non-2xx response returns the same shape:

```
json
```

```

{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "One or more fields are invalid.",
    "details": [
      { "field": "email", "issue": "Must be a valid email address." }
    ]
  }
}

```

HTTP Status	When
400	Validation failure, malformed request
401	Missing/expired/invalid token
403	Authenticated, but role/ownership forbids the action
404	Resource doesn't exist or belongs to another tenant (never leak existence)
409	Conflict (e.g., duplicate enrollment, quota already at limit)
422	Business-rule violation (e.g., publishing an exam with zero questions)
429	Rate limit exceeded
500	Unhandled server error

Payment-specific `error.code` values (all returned as 422 unless noted):

`PAID_CLASSROOM_REQUIRES_CHECKOUT`, `CLASSROOM_AT_CAPACITY`, `PAYMENT_ALREADY_PENDING` (student already has an unresolved transaction for this classroom), `PAYMENT_FAILED`.

1.5 Versioning

URI-versioned (`/api/v1/...`). Breaking changes ship as `/api/v2/...`; additive changes (new optional fields) do not bump the version.

1.6 Common DTOs (referenced across modules)

- `ErrorResponse` — see 1.4.
- `PagedResult<T>` — see 1.3.
- `UserSummaryDto`

Field	Type	Notes
<code>userId</code>	guid	
<code>fullName</code>	string	
<code>role</code>	string	<code>Teacher Student SuperAdmin</code>

2. Module: Auth

#	Method	Path	Auth	Purpose
2.1	POST	<code>/auth/register/teacher</code>	None	Teacher self-signup
2.2	POST	<code>/auth/register/student</code>	None	Student self-signup
2.3	POST	<code>/auth/login</code>	None	Login, returns access + refresh token
2.4	POST	<code>/auth/refresh-token</code>	None (refresh token in body)	Exchange refresh token for new access token
2.5	POST	<code>/auth/logout</code>	Bearer	Revokes the current refresh token
2.6	GET	<code>/auth/me</code>	Bearer	Returns the caller's own profile
2.7	POST	<code>/auth/password-reset/request</code>	None	Request a password reset link
2.8	POST	<code>/auth/password-reset/confirm</code>	None (token in body)	Set a new password using a valid reset token

2.1 POST `/auth/register/teacher`

Request — `RegisterTeacherRequest`

Field	Type	Required	Notes
<code>email</code>	string	Yes	Must be unique
<code>password</code>	string	Yes	Min 8 chars, 1 number, 1 uppercase
<code>fullName</code>	string	Yes	
<code>phone</code>	string	No	

Response `201` — `AuthResponse`

Field	Type	Notes
<code>accessToken</code>	string	JWT
<code>refreshToken</code>	string	opaque token
<code>expiresIn</code>	int	seconds
<code>user</code>	UserSummaryDto	

Errors: `400` invalid password/email format, `409` email already registered.

2.2 POST /auth/register/student

Request — `RegisterStudentRequest`

Field	Type	Required	Notes
<code>email</code>	string	Yes	Unique
<code>password</code>	string	Yes	Same policy as teacher
<code>fullName</code>	string	Yes	
<code>parentGuardianEmail</code>	string	Yes	Required per business rule — student profile always has exactly one guardian email
<code>dateOfBirth</code>	date	No	

Response `201`: `AuthResponse` (same shape as above). Errors: `400`, `409`.

2.3 POST /auth/login

Request — `LoginRequest`: `email` (string, required), `password` (string, required). Response `200`: `AuthResponse`. Errors: `401` invalid credentials, `403` account deactivated.

2.4 POST /auth/refresh-token

Request: `refreshToken` (string, required). Response `200`: `AuthResponse` (new pair). Errors: `401` refresh token expired/revoked.

2.5 POST /auth/logout

Request: empty body. Response `204`.

2.6 GET /auth/me

Response `200` — role-dependent:

- Teacher → `TeacherProfileDto` (`userId`, `email`, `fullName`, `phone`, `currentPlan`: `SubscriptionPlanSummaryDto`)
- Student → `StudentProfileDto` (`userId`, `email`, `fullName`, `parentGuardianEmail`, `dateOfBirth`)

2.7 POST /auth/password-reset/request

Public — no auth token required.

Request — `PasswordResetRequestDto`

Field	Type	Required	Notes
<code>email</code>	string	Yes	Any email format, existing or not

Response `200` — always returns the same response, regardless of whether the email is registered, to prevent user enumeration:

```
json
{ "message": "If that email exists, a reset link has been sent." }
```

Behavior: if the email matches an active account, generates a single-use, time-limited reset token (recommended TTL: 30-60 minutes) and emails a reset link containing it. If the email doesn't match any account, nothing is sent, but the response is identical — the client can never distinguish the two cases.

Errors: `400` only for malformed email syntax. Never `404`.

2.8 POST /auth/password-reset/confirm

Public — authenticated by the reset token itself, not a JWT.

Request — `PasswordResetConfirmDto`

Field	Type	Required	Notes
<code>token</code>	string	Yes	From the emailed reset link
<code>newPassword</code>	string	Yes	Same policy as registration (min 8 chars, 1 number, 1 uppercase)

Response `200`:

```
json
{ "message": "Password has been reset." }
```

Behavior: validates the token (exists, unexpired, unused), updates the password, marks the token used (single-use — replay with the same token fails), and revokes all existing refresh tokens for that user (forces re-login everywhere, standard practice after a password change). Does **not** auto-issue a new access token — the user logs in normally with the new password via 2.3.

Errors: `400` invalid, expired, or already-used token; `400` new password fails policy.

3. Module: Classrooms & Enrollment

#	Method	Path	Auth	Purpose
3.1	GET	<code>/subjects</code>	Bearer (any)	Lookup list for classroom creation
3.2	POST	<code>/classrooms</code>	Teacher	Create a classroom
3.3	GET	<code>/classrooms</code>	Teacher, Student	Teacher: classrooms they own. Student: classrooms they're enrolled in
3.4	GET	<code>/classrooms/{classroomId}</code>	Teacher (owner), Student (enrolled)	Classroom detail
3.5	PUT	<code>/classrooms/{classroomId}</code>	Teacher (owner)	Update name/subject/active status
3.6	DELETE	<code>/classrooms/{classroomId}</code>	Teacher (owner)	Soft- delete/deactivate
3.7	POST	<code>/classrooms/{classroomId}/regenerate-code</code>	Teacher (owner)	Invalidate old enrollment code, issue new
3.8	POST	<code>/classrooms/enroll</code>	Student	Join a free classroom via enrollment code (instant) — see Module 3A for paid classrooms
3.9	GET	<code>/classrooms/{classroomId}/students</code>	Teacher (owner)	Roster
3.10	DELETE	<code>/classrooms/{classroomId}/students/{studentId}</code>	Teacher (owner)	Remove a student
3.11	PUT	<code>/classrooms/{classroomId}/pricing</code>	Teacher (owner)	Set/update the classroom's price ($\emptyset$ = free)

#	Method	Path	Auth	Purpose
3.12	GET	<code>/classrooms/{classroomId}/pricing</code>	Teacher (owner), Student	Get current price before attempting checkout

3.2 POST `/classrooms`

Request — `CreateClassroomRequest`

Field	Type	Required
<code>subjectId</code>	guid	Yes
<code>name</code>	string	Yes

Response `201` — `ClassroomDto`

Field	Type	Notes
<code>classroomId</code>	guid	
<code>teacherId</code>	guid	
<code>subjectName</code>	string	
<code>name</code>	string	
<code>enrollmentCode</code>	string	
<code>isActive</code>	bool	
<code>studentCount</code>	int	
<code>createdAt</code>	datetime	

Errors: `422` if the `TeacherSubscription` plan's `MaxStudents`/classroom limits would be violated (checked at enrollment time too, see 3.8).

3.8 POST `/classrooms/enroll`

Request: `enrollmentCode` (string, required). Response `200`: `ClassroomDto` (the joined classroom). Errors: `404` invalid code, `409` already enrolled, `422` classroom's teacher has hit `MaxStudents` on their plan, `422 PAID_CLASSROOM_REQUIRES_CHECKOUT` if the classroom has a non-zero price — the client must redirect to the Module 3A checkout flow instead.

3.9 GET /classrooms/{classroomId}/students

Response `200`: `PagedResult<StudentRosterItemDto>` — `{ studentId, fullName, enrolledAt, status }`.

3.11 PUT /classrooms/{classroomId}/pricing

Request — `SetClassroomPricingRequest`

Field	Type	Required	Notes
<code>price</code>	decimal	Yes	<code>0</code> marks the classroom free — existing <code>/classrooms/enroll</code> flow applies
<code>currency</code>	string	No	Defaults to <code>EGP</code> ; only <code>EGP</code> supported currently

Response `200`: `ClassroomPricingDto` (`classroomId`, `price`, `currency`, `isFree`, `updatedAt`). Note: re-pricing never affects students already enrolled, or in-flight `PaymentTransaction`s — those keep the price snapshotted at the time they paid.

3.12 GET /classrooms/{classroomId}/pricing

Response `200`: `ClassroomPricingDto`.

3A. Module: Payments (Paymob)

#	Method	Path	Auth	Purpose
3A.1	POST	<code>/classrooms/{classroomId}/enrollment-checkout</code>	Student	Initiate a Paymob payment intent for a paid classroom
3A.2	GET	<code>/enrollment-checkout/{transactionId}/status</code>	Student (own transaction)	Poll payment/enrollment status after returning from Paymob
3A.3	POST	<code>/webhooks/payments/paymob</code>	None (HMAC-signed instead)	Server-to-server confirmation from Paymob — the only thing allowed to create a paid Enrollment
3A.4	GET	<code>/teacher/payouts</code>	Teacher	List own payout statements
3A.5	GET	<code>/teacher/payouts/{payoutId}</code>	Teacher (own)	Payout statement detail
3A.6	GET	<code>/admin/payouts</code>	SuperAdmin	All teachers' payout statements (finance ops)
3A.7	PUT	<code>/admin/payouts/{payoutId}/mark-paid</code>	SuperAdmin	Mark a payout as transferred

3A.1 POST `/classrooms/{classroomId}/enrollment-checkout`

Request: empty body (price is read server-side from `ClassroomPricing`, never trusted from the client). Response `201` — `CheckoutSessionDto`

Field	Type	Notes
<code>transactionId</code>	guid	Draya's internal <code>PaymentTransaction.Id</code>
<code>paymentUrl</code>	string	Hosted Paymob checkout page to redirect the student to
<code>amount</code>	decimal	Snapshoted from <code>ClassroomPricing.Price</code>
<code>currency</code>	string	
<code>status</code>	string	Always <code>Pending</code> at creation

Errors: `404` classroom not found, `409` already enrolled, `422` classroom is free (use 3.8 instead), `422` classroom at capacity.

3A.2 GET `/enrollment-checkout/{transactionId}/status`

Response `200`:

```

json
{
  "transactionId": "...",
  "status": "Paid",
  "enrollmentId": "...",
  "classroomId": "..."
}

```

`status` is one of `Pending` | `Paid` | `Failed` | `Refunded`. `enrollmentId` is null until status becomes `Paid`. This endpoint never triggers grading of the payment itself — it only reflects what the webhook has already recorded.

3A.3 POST `/webhooks/payments/paymob`

Not part of the public API surface — called only by Paymob's servers. Documented here because frontend/mobile need to know it exists and why polling (3A.2) is required instead of trusting any client-side "success" redirect.

Request: raw Paymob webhook payload (varies by event type) plus an HMAC signature header. Behavior:

1. Verify signature against the shared secret. Invalid → log to `PaymentWebhookLog` with `SignatureValid: false`, return `400`, do nothing else.
2. Look up `PaymentTransaction` by `GatewayTransactionId`. Not found → log `TransactionNotFound`, return `200` (per Paymob's retry semantics — a `200` tells them to stop retrying an event we'll never be able to match).
3. Idempotency check: if `Status` is already `Paid`, log `DuplicateIgnored`, return `200` immediately.
4. Otherwise: mark `Paid`, set `PaidAt`, create the `Enrollment` row (re-checking capacity inside the same transaction), send an `EnrollmentConfirmed` notification, log `Processed`.

Response: always (200) on any successfully handled case (including duplicates/unmatched) so Paymob stops retrying; (400) only for signature failures.

3A.4 GET /teacher/payouts

Response (200): PagedResult<TeacherPayoutStatementDto> — { payoutId, periodStart, periodEnd, grossAmount, platformFeePercent, platformFeeAmount, netAmount, status, paidAt }.

3A.7 PUT /admin/payouts/{payoutId}/mark-paid

Request: empty body. Response (200): updated TeacherPayoutStatementDto with status: "Paid", paidAt set. This is a manual confirmation step (bank transfer happens outside the platform) — not itself a payment gateway call.

4. Module: Materials

#	Method	Path	Auth	Purpose
4.1	POST	/classrooms/{classroomId}/materials	Teacher (owner)	Upload new lesson (creates Material + v1)
4.2	GET	/classrooms/{classroomId}/materials	Teacher (owner), Student (enrolled)	List materials
4.3	GET	/materials/{materialId}	Teacher (owner), Student (enrolled)	Material detail incl. current version
4.4	POST	/materials/{materialId}/versions	Teacher (owner)	Upload a new version of an existing material
4.5	GET	/materials/{materialId}/versions	Teacher (owner)	Full version history
4.6	GET	/materials/{materialId}/versions/{versionId}/status	Teacher (owner)	Poll parse status (Pending/Parsed/Failed)
4.7	DELETE	/materials/{materialId}	Teacher (owner)	Soft delete
4.8	GET	/materials/{materialId}/stream	Teacher (owner), Student (enrolled)	Returns a signed streaming URL (video only)

4.1 POST /classrooms/{classroomId}/materials

Request: `multipart/form-data`

Field	Type	Required	Notes
<code>title</code>	string	Yes	
<code>materialType</code>	string	Yes	<code>PDF</code> <code>DOCX</code> <code>PPTX</code> <code>Video</code> <code>Image</code>
<code>file</code>	binary	Yes	Max size enforced by teacher's plan (<code>MaxStorageMB</code>)

Response `202 Accepted` — `MaterialDto`

Field	Type	Notes
<code>materialId</code>	guid	
<code>title</code>	string	
<code>materialType</code>	string	
<code>currentVersion</code>	<code>MaterialVersionDto</code>	see below
<code>createdAt</code>	datetime	

`202`, not `201` — parsing/chunking happens asynchronously; the client polls `4.6` or listens for a `MaterialParsed` notification (Module 8).

`MaterialVersionDto`

Field	Type	Notes
<code>versionId</code>	guid	
<code>versionNumber</code>	int	
<code>fileUrl</code>	string	
<code>parseStatus</code>	string	<code>Pending</code> <code>Parsed</code> <code>Failed</code>
<code>uploadedAt</code>	datetime	

Errors: `422` storage quota exceeded, `415` unsupported file type.

4.4 POST /materials/{materialId}/versions

Same request shape as 4.1 minus `title`/`materialType`. Response `202`:

`MaterialVersionDto`. Existing exams keep referencing the prior version automatically — this endpoint never mutates past exams.

5. Module: Exams & Question Bank

#	Method	Path	Auth	Purpose
5.1	POST	<code>/classrooms/{classroomId}/exams/generate</code>	Teacher (owner)	AI-generate a draft exam from a material version
5.2	POST	<code>/exams</code>	Teacher	Create an empty/manual exam
5.3	GET	<code>/classrooms/{classroomId}/exams</code>	Teacher (owner), Student (enrolled, published only)	List exams
5.4	GET	<code>/exams/{examId}</code>	Teacher (owner), Student (enrolled, published only)	Exam detail with questions
5.5	PUT	<code>/exams/{examId}</code>	Teacher (owner)	Update exam metadata (title, time limit, randomization)
5.6	POST	<code>/exams/{examId}/questions</code>	Teacher (owner)	Attach a question (from bank or newly authored) to the exam
5.7	DELETE	<code>/exams/{examId}/questions/{questionId}</code>	Teacher (owner)	Remove a question from the exam
5.8	PUT	<code>/exams/{examId}/questions/reorder</code>	Teacher (owner)	Bulk reorder / repoint values
5.9	POST	<code>/exams/{examId}/publish</code>	Teacher (owner)	Publish (Draft → Published)
5.10	GET	<code>/question-bank</code>	Teacher	List own questions (filter by subject/type/difficulty)

#	Method	Path	Auth	Purpose
5.11	POST	<code>/question-bank</code>	Teacher	Manually author a question
5.12	PUT	<code>/questions/{questionId}</code>	Teacher (owner)	Edit a question / its rubric
5.13	DELETE	<code>/questions/{questionId}</code>	Teacher (owner)	Deactivate a question
5.14	POST	<code>/exams/{examId}/attempts/start</code>	Student (enrolled)	Start an attempt
5.15	POST	<code>/attempts/{attemptId}/answers</code>	Student (own attempt)	Autosave an answer
5.16	POST	<code>/attempts/{attemptId}/submit</code>	Student (own attempt)	Final submit → triggers grading
5.17	POST	<code>/attempts/{attemptId}/anti-cheating-events</code>	Student (own attempt)	Report a detected violation

5.1 POST `/classrooms/{classroomId}/exams/generate`

Request — `GenerateExamRequest`

Field	Type	Required	Notes
<code>materialVersionId</code>	guid	Yes	Source content
<code>difficultyLevel</code>	string	Yes	<code>Easy</code> <code>Medium</code> <code>Hard</code>
<code>questionTypes</code>	string[]	Yes	Any of <code>MCQ</code> , <code>TrueFalse</code> , <code>FillBlank</code> , <code>ShortAnswer</code> , <code>Essay</code>
<code>questionCount</code>	int	Yes	Requested count — actual may be lower, see <code>DATA_UNAVAILABLE</code> note
<code>title</code>	string	No	Defaults to material title

Response `202 Accepted` — `ExamGenerationJobDto`

Field	Type	Notes
<code>jobId</code>	guid	Poll via <code>GET /exam-generation-jobs/{jobId}</code> (see note)
<code>status</code>	string	<code>Queued</code> <code>Processing</code> <code>Completed</code> <code>PartiallyCompleted</code> <code>Failed</code>

Design note: Generation is asynchronous (LLM calls take seconds). On `Completed`/`PartiallyCompleted`, the job result includes `examId` and, if fewer questions were produced than requested, an `insufficientContentWarning: true` flag (this is the `DATA_UNAVAILABLE` guardrail surfacing to the teacher, per agreed grading-safety rule) plus `generatedCount` vs `requestedCount`.

Errors: `422` monthly exam-generation quota exceeded (`UsageCounter`).

5.6 POST /exams/{examId}/questions

Request — `AddQuestionToExamRequest`

Field	Type	Required	Notes
<code>questionId</code>	guid	Yes (if reusing bank question)	mutually exclusive with <code>newQuestion</code>
<code>newQuestion</code>	QuestionCreateDto	Yes (if authoring inline)	see 5.11
<code>points</code>	decimal	Yes	
<code>orderIndex</code>	int	No	Appended to end if omitted

Response `201`: `ExamQuestionDto` (`examId`, `questionId`, `orderIndex`, `points`, embedded `QuestionDto`).

5.9 POST /exams/{examId}/publish

Response `200`: `ExamDto` with `status: "Published"`, `publishedAt` set. Errors: `422` — exam has zero questions, or an Essay/ShortAnswer question is missing a rubric.

5.11 POST /question-bank — QuestionCreateDto

Field	Type	Required	Notes
<code>subjectId</code>	guid	No	
<code>questionType</code>	string	Yes	<code>MCQ</code> <code>TrueFalse</code> <code>FillBlank</code> <code>ShortAnswer</code> <code>Essay</code>
<code>questionText</code>	string	Yes	
<code>difficultyLevel</code>	string	Yes	<code>Easy</code> <code>Medium</code> <code>Hard</code>
<code>options</code>	QuestionOptionDto[]	Required if MCQ/TrueFalse	<code>{ optionText, isCorrect, orderIndex }</code> — exactly one <code>isCorrect: true</code> enforced server-side
<code>rubric</code>	RubricCreateDto	Optional if Essay/ShortAnswer	<code>{ criteria, teacherInstructions }</code> — if omitted for these types, AI proposes one on save

Response `201`: `QuestionDto`.

5.14 POST /exams/{examId}/attempts/start

Response `201`: `ExamAttemptDto`

Field	Type	Notes
<code>attemptId</code>	guid	
<code>examId</code>	guid	
<code>startedAt</code>	datetime	
<code>timeLimitMinutes</code>	int?	
<code>maxScore</code>	decimal	Snapshot of total points at start time
<code>questions</code>	ExamAttemptQuestionDto[]	Randomized per <code>RandomizeQuestionOrder</code> / <code>RandomizeOptionOrder</code> ; correct answers and rubrics omitted

Errors: (409) attempt already exists for this student+exam (one-attempt policy — flagged as an assumption in the DB design; revisit if retakes are needed).

5.15 POST /attempts/{attemptId}/answers

Request — (SubmitAnswerRequest): (questionId) (guid, required), (selectedOptionId) (guid, required for MCQ/TrueFalse), (answerText) (string, required for FillBlank/ShortAnswer/Essay). Response (200): { questionId, saved: true } — this is an autosave, no grading happens yet.

5.16 POST /attempts/{attemptId}/submit

Response (202 Accepted) — grading is asynchronous (AI grading call + rubric evaluation):

json

```
{ "attemptId": "...", "status": "Submitted", "gradingStatus": "Processing" }
```

Client polls (GET /attempts/{attemptId}/results) (Module 6) or listens for a (ResultsPublished) notification.

5.17 POST /attempts/{attemptId}/anti-cheating-events

Request: (eventType) (string, required: (TabSwitch | CopyPaste | AppMinimized | FocusLoss)). Response (200):

json

```
{ "sequenceNumber": 2, "action": "Flagged" }
```

(action) is (Warning) on the first event, (Flagged) from the second onward (fixed platform-wide rule).

6. Module: Grading

#	Method	Path	Auth	Purpose
6.1	GET	/attempts/{attemptId}/results	Student (own), Teacher (owner)	Full graded result
6.2	GET	/exams/{examId}/attempts	Teacher (owner)	All students' attempts + scores for an exam
6.3	GET	/answers/{answerId}	Teacher (owner)	Single answer detail incl. AI confidence
6.4	PUT	/answers/{answerId}/override	Teacher (owner)	Override an AI- assigned score
6.5	GET	/answers/{answerId}/overrides	Teacher (owner)	Override history (audit trail)

6.1 GET /attempts/{attemptId}/results

Response 200 — ExamAttemptResultDto

Field	Type	Notes
attemptId	guid	
status	string	Submitted Flagged
totalScore	decimal	
maxScore	decimal	
gradedAt	datetime	
flaggedForReview	bool	true if 2+ anti-cheating events occurred
answers	StudentAnswerResultDto[]	{ questionId, aiScore, finalScore, confidenceScore, wasOverridden }

Errors: 409 grading still in progress (gradingStatus: Processing).

6.4 PUT /answers/{answerId}/override

Request — OverrideGradeRequest

Field	Type	Required
<code>newScore</code>	decimal	Yes
<code>reason</code>	string	No

Response `200`: `StudentAnswerDto` with updated `finalScore`. Server writes an immutable `GradeOverride` row internally — never overwrites history (client can retrieve it via 6.5).
 Errors: `422` `newScore` exceeds the question's max points.

7. Module: Reports

#	Method	Path	Auth	Purpose
7.1	GET	<code>/students/{studentId}/reports</code>	Teacher (of that student), Student (own)	List AI-generated performance/weakness reports
7.2	GET	<code>/reports/{reportId}</code>	Teacher (of that student), Student (own)	Full report detail
7.3	GET	<code>/classrooms/{classroomId}/reports/overview</code>	Teacher (owner)	Aggregated class-level weak-topic analytics
7.4	GET	<code>/students/{studentId}/parent-report-logs</code>	Teacher (of that student)	Audit view: which emails were sent to the guardian, and when

7.1 GET `/students/{studentId}/reports`

Response `200`: `PagedResult<PerformanceReportSummaryDto>` — `{ reportId, examAttemptId, generatedAt, weakTopicsCount }`.

7.2 GET `/reports/{reportId}`

Response `200` — `PerformanceReportDto`

Field	Type	Notes
reportId	guid	
studentId	guid	
examAttemptId	guid	
summaryText	string	AI-generated narrative summary
weakTopics	WeakTopicDto[]	{ topic, proficiencyScore, recommendation }
generatedAt	datetime	

Resolved (see 0.1 history): the original pitch doc described a teacher-approval gate (`IsApproved`) before a report becomes visible to parents. The later clarification round confirmed parent emails are sent automatically and immediately after every exam, with no linking/approval workflow for parents at all — no parent accounts exist. This contract reflects that ruling: reports/emails go out immediately, no approval step. The teacher-facing report (this endpoint) is always visible to the teacher immediately regardless.

7.3 GET /classrooms/{classroomId}/reports/overview

Response `200`:

```
json
{
  "classroomId": "...",
  "studentCount": 28,
  "weakestTopics": [
    { "topic": "Quadratic Equations", "avgProficiency": 0.42, "studentsBelow60P"
  ]
}
```

8. Additional Modules (brief — not in this sprint's core scope, listed for completeness)

Module	Endpoints
Notifications	<code>GET /notifications</code> , <code>PUT /notifications/{id}/read</code>
Chat	<code>GET /classrooms/{classroomId}/messages</code> , <code>POST /classrooms/{classroomId}/messages</code>
Subscriptions (Teacher-facing)	<code>GET /subscription/current</code> , <code>GET /subscription/usage</code>
Admin (SuperAdmin only)	<code>GET/POST/PUT /admin/subscription-plans</code> , <code>GET /admin/teachers</code> , <code>PUT /admin/teachers/{id}/deactivate</code>

These will get full DTO documentation in a follow-up pass once the six core modules above are signed off — flagging now so frontend/mobile know they exist but aren't blocking on them yet.

9. Swagger / Postman Setup

The backend will expose a live OpenAPI 3.0 document at `/swagger/v1/swagger.json`, rendered at `/swagger` via Swagger UI (`Swashbuckle.AspNetCore`), auto-generated from controller/DTO annotations — this is the source of truth once code exists; this markdown document is the source of truth before code exists.

A companion static `draya-api.yaml` (delivered alongside this document) can be imported directly into Postman (`Import → File`) to generate a full request collection with folders per module, or into any standalone Swagger UI/editor.

Postman environment variables to define: `baseUrl`, `accessToken`, `refreshToken` — set `accessToken` via a login request's test script (`pm.environment.set(...)`) so subsequent requests auto-attach the Bearer header.

Recommended Postman collection structure mirrors Sections 2–7 above (one folder per module).

10. Review Checklist (for frontend/mobile sign-off meeting)

- Confirm the one-attempt-per-exam assumption (Section 5.14) — no retake flow needed?
- ~~Resolve the parent report approval flag (Section 7.2)~~ — **resolved**: immediate send, no approval gate.
- Confirm async patterns (`202` + polling/notification) for exam generation, material parsing, and grading are acceptable for mobile UX, or whether a WebSocket/SignalR

push is preferred instead of polling.

- Confirm error code taxonomy (Section 1.4) covers all cases frontend needs to branch on (e.g., distinguishing "quota exceeded" from generic 422s — should quota errors get their own code value like `QUOTA_EXCEEDED`?).
- Confirm file upload size limits should be enforced client-side too (mirroring `MaxStorageMB`) for better UX before upload starts.
- Sign off on DTO field naming convention (`camelCase`) throughout, as shown).
- Confirm the two payment assumptions (Section 0.1): Draya as merchant of record with commission, and one-time (not recurring) payment per classroom.
- Decide the refund trigger policy (capacity race, teacher deletes classroom, student dispute) — currently modeled as a `Refunded` status but the workflow that sets it isn't built yet.
- Confirm whether the payment webhook path (`/webhooks/payments/paymob`) needs to sit outside standard rate-limiting/auth middleware entirely, given it's authenticated by HMAC signature rather than JWT.
- **New:** Confirm the password-reset email delivery mechanism (Section 2.7) — does it reuse the Module 8 email notification service, or is it a dedicated transactional email flow kept separate for deliverability/security reasons?
- ~~Resolve the ownership error code ambiguity (Section 1.1)~~ — **resolved:** always 404 for cross-tenant access.

End of API contract document. Once reviewed and confirmed, this becomes the frozen v1 contract for parallel frontend/mobile/backend development.